

```

import os
import time
from playwright.sync_api import sync_playwright

def automate_social_media_post(platform, username, password,
                                post_content, image_path=None):
    """
    Automates logging into a social media platform and publishing a
    post using Playwright.
    Supported platforms in this template: 'twitter' (X) and
    'linkedin'.

    :param platform: Target social media platform ('twitter' or
    'linkedin')
    :param username: Account username or email
    :param password: Account password
    :param post_content: Text content of the post
    :param image_path: Optional local path to an image file to attach
    """
    print(f"Initializing browser automation for
    {platform.capitalize()}...")

    # Launch browser in headful mode (headless=False) so you can see
    the automation happen
    # and handle any unexpected security checks or captchas manually
    if they arise.
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=False,
        args=["--start-maximized"])
        # Create a browser context with standard viewport matching a
        real desktop browser
        context = browser.new_context(viewport={"width": 1280,
        "height": 720})
        page = context.new_page()

        try:
            if platform.lower() == 'twitter' or platform.lower() ==
            'x':
                print("Navigating to X/Twitter Login...")
                page.goto("https://x.com/i/flow/login")

                # Fill in Username/Email
                # X loads dynamically; wait for the input selector to
                become visible

                page.wait_for_selector("input[autocomplete='username']")
                page.fill("input[autocomplete='username']", username)
                page.click("text=Next")

```

```

        # Sometimes X prompts for an unusual activity
verification (phone number/handle)
        # This pause handles unexpected transition gaps
cleanly
        time.sleep(2)

        # Check if password field is ready, or if an
intermediary step popped up
        if page.locator("input[name='password']").count() ==
0:
            # If it asks for username verification again due
to suspicious login detection

page.fill("input[data-testid='ocfEnterTextTextInput']", username)
        page.click("text=Next")

        # Fill in Password
page.wait_for_selector("input[name='password']")
page.fill("input[name='password']", password)
page.click("data-testid=LoginForm_Login_Button")

print("Login submitted. Waiting for home feed...")
page.wait_for_url("https://x.com/home", timeout=60000)

        # Locate the compose box text area

page.wait_for_selector("div[data-testid='tweetTextarea_0']")
        page.click("div[data-testid='tweetTextarea_0']")
        page.fill("div[data-testid='tweetTextarea_0']",
post_content)

        # Handle image upload if provided
if image_path and os.path.exists(image_path):
    print(f"Uploading media: {image_path}")
    # Target the hidden file input element used for
file uploads
        file_input =
page.locator("input[data-testid='fileInput']")
        file_input.set_input_files(image_path)
        time.sleep(2) # Give media time to render/compress

        # Click the Tweet/Post button
print("Publishing post...")
page.click("data-testid=tweetButtonInline")
        time.sleep(4) # Allow structural network requests to
complete
print("Post successfully published on X/Twitter!")

```

```

elif platform.lower() == 'linkedin':
    print("Navigating to LinkedIn Login...")
    page.goto("https://www.linkedin.com/login")

    # Fill login credentials
    page.fill("#username", username)
    page.fill("#password", password)
    page.click("button[type='submit']")

    print("Login submitted. Waiting for feed...")
    page.wait_for_url("https://www.linkedin.com/feed/",
timeout=60000)

    # Click the "Start a post" element to launch modal
editor

page.wait_for_selector("button.share-box-feed-entry__trigger")
page.click("button.share-box-feed-entry__trigger")

    # Wait for the text areas inside the editor modal to
be interactable
page.wait_for_selector(".ql-editor")
page.fill(".ql-editor", post_content)

    # Handle image upload if provided
if image_path and os.path.exists(image_path):
    print(f"Uploading media: {image_path}")
    # Click image icon to trigger file dialog hook
    page.click("button[aria-label='Add media']")
    # Target the file selector injected into theDOM
    file_input = page.locator("input[type='file']")
    file_input.set_input_files(image_path)
    # Complete media setup steps (LinkedIn requires
clicking "Next" on the image preview modal)
    page.click("button:has-text('Next')")
    time.sleep(2)

    # Click the final Post button
    print("Publishing post...")
    page.click(".share-actions__primary-action")
    time.sleep(4)
    print("Post successfully published on LinkedIn!")

else:
    print(f"Unsupported platform: {platform}")

except Exception as e:

```

```

        print(f"\n[!] Automation failed: {e}")
        print("Note: Social media platforms frequently change UI
elements. Verify selectors if issues persist.")
    finally:
        # Cleanly close connection instances
        browser.close()
        print("Browser closed.")

if __name__ == "__main__":
    # --- Example Configuration Execution ---
    # Setup your target metrics below
    target_platform = input("Enter platform (twitter/linkedin):
").strip().lower()
    acc_user = input("Enter account email/username: ").strip()
    acc_pass = input("Enter account password: ").strip()
    content = input("Enter post text content: ").strip()
    img = input("Enter local path to image (optional, press Enter to
skip): ").strip()

    if target_platform and acc_user and acc_pass and content:
        automate_social_media_post(
            platform=target_platform,
            username=acc_user,
            password=acc_pass,
            post_content=content,
            image_path=img if img else None
        )
    else:
        print("Missing required fields. Automation aborted.")

```